

HEAPBEAT

KỊCH BẢN THUYẾT TRÌNH CHÍNH THỨC

Đề tài: PHÁT NHẠC CHỜ CỘNG ĐỒNG CHO PHÒNG TỰ HỌC
(COLLABORATIVE DJ QUEUE)

Người thuyết trình: PHẠM HÙNG TIỀN - MSSV 24207043

Ngày thuyết trình: 08/08/2026

Thời lượng: 15 phút báo cáo và 5 phút trình diễn

I. PHÂN BỐ THỜI GIAN

00:00 - 00:40 Slide 1 Giới thiệu đề tài
00:40 - 01:50 Slide 2 Bối cảnh và yêu cầu hệ thống
01:50 - 02:45 Slide 3 Ảnh xạ chức năng sang cấu trúc dữ liệu
02:45 - 04:15 Slide 4 Max-Heap và Upvote/Downvote
04:15 - 05:20 Slide 5 Danh sách liên kết đôi vòng
05:20 - 06:50 Slide 6 Hash Map chống spam và thu hồi bài
06:50 - 07:50 Slide 7 Kiến trúc và luồng trạng thái
07:50 - 09:20 Slide 8 Luồng yêu cầu, bình chọn và phát bài
09:20 - 10:10 Slide 9 Kiểm soát quyền phát
10:10 - 11:00 Slide 10 Giao diện quan sát cấu trúc dữ liệu
11:00 - 12:20 Slide 11 Kiểm thử và kết quả
12:20 - 13:40 Slide 12 Lý do lựa chọn Max-Heap
13:40 - 15:00 Slide 13 Kết luận và chuyển sang trình diễn
15:00 - 20:00 Trình diễn hệ thống

II. NỘI DUNG THUYẾT TRÌNH

SLIDE 1 - GIỚI THIỆU - 40 GIÂY

Kính chào thầy cô và các bạn. Em là Phạm Hùng Tiến, đại diện nhóm trình bày đề án HeapBeat với đề tài Phát nhạc chờ cộng đồng cho phòng tự học, hay Collaborative DJ Queue. Hệ thống mô phỏng việc phát nhạc tại các không gian chung của trường như sảnh và phòng tự học. Sinh viên có thể gửi bài và bình chọn; hệ thống tự quyết định thứ tự phát bằng Max-Heap, danh sách liên kết đôi vòng và Hash Map.

SLIDE 2 - BỐI CẢNH VÀ YÊU CẦU HỆ THỐNG - 70 GIÂY

Trong một không gian chung, nhiều sinh viên cùng nghe một loa nhưng có sở thích khác nhau. Nếu chỉ phát theo thứ tự gửi, người gửi sớm luôn chiếm ưu thế. Nếu cho gửi tự do, một tài khoản có thể spam cùng một bài hoặc chiếm toàn bộ hàng đợi. HeapBeat giải quyết đồng thời bốn yêu cầu.

Thứ nhất, bài có mức bình chọn cao nhất luôn là bài tiếp theo. Thứ hai, trình phát hỗ trợ Next, Previous, Repeat All và thêm bài mới vào cuối danh sách phát. Thứ ba, Upvote hoặc Downvote làm thứ hạng thay đổi ngay mà không sắp xếp lại toàn bộ danh sách. Thứ tư, nếu một StudentID gửi yêu cầu thứ tư trong mười phút

hoặc gửi trùng bài, tài khoản bị chặn ba mươi phút và toàn bộ bài đang chờ của tài khoản đó bị thu hồi.

SLIDE 3 - ÁNH XẠ CHÚC NĂNG SANG CẤU TRÚC DỮ LIỆU - 55 GIÂY

Hàng đợi bình chọn sử dụng Max-Heap. Phần tử ở gốc là bài có độ ưu tiên cao nhất; Map từ requestId sang index giúp định vị nhanh bài vừa được bình chọn.

Lịch sử phát sử dụng Circular Doubly Linked List. Liên kết hai chiều hỗ trợ Next và Previous trong $O(1)$; liên kết từ đuôi về đầu tạo Repeat All; thao tác addLast thêm bài mới vào cuối danh sách.

SpamGuard sử dụng Hash Map theo StudentID. Mỗi giá trị lưu các thời điểm gửi trong cửa sổ mười phút, tập bài đã gửi, các requestId đang sở hữu và thời điểm hết chặn. Nhờ đó hệ thống có thể từ chối vi phạm và thu hồi chính xác các bài liên quan.

SLIDE 4 - MAX-HEAP VÀ BÌNH CHỌN - 90 GIÂY

Max-Heap được lưu bằng mảng. Với nút i , node cha nằm tại floor của $(i \text{ trừ } 1)$ chia 2; hai node con nằm tại $2i$ cộng 1 và $2i$ cộng 2. Bất biến là node cha không có độ ưu tiên thấp hơn node con.

Khi sinh viên Upvote, điểm tăng và phần tử có thể vi phạm bất biến với cha, nên hệ thống gọi heapifyUp. Khi Downvote, điểm giảm và phần tử có thể thấp hơn node con, nên gọi heapifyDown. Map requestId sang index cho phép tìm vị trí hiện tại trong $O(1)$ trung bình; tái cân bằng chỉ đi theo chiều cao cây nên có chi phí $O(\log n)$.

Bộ so sánh xét lần lượt điểm vote, số lượt upvote, thứ tự do quản trị viên đặt và thời điểm gửi. Quy tắc này tạo thứ tự xác định ngay cả khi hai bài bằng điểm.

SLIDE 5 - DANH SÁCH LIÊN KẾT ĐÔI VÒNG - 65 GIÂY

Mỗi node trong danh sách phát chứa bài hát, con trỏ prev và con trỏ next. Nút đầu có prev trỏ về đuôi; nút đuôi có next trỏ về đầu. Vì vậy current-next thực hiện Next, current-prev thực hiện Previous và toàn bộ danh sách tự khép vòng.

Khi một bài rời hàng đợi để phát, addLast nối bài đó vào cuối lịch sử. Trường hợp danh sách rỗng, một node hoặc nhiều node đều dùng cùng một mô hình liên kết; các thao tác di chuyển và thêm cuối có chi phí $O(1)$.

SLIDE 6 - HASH MAP CHỐNG SPAM - 90 GIÂY

SpamGuard ánh xạ mỗi StudentID tới một trạng thái riêng. Trước khi thay đổi hàng đợi, hệ thống loại các mốc cũ hơn mười phút, kiểm tra blockedUntil, kiểm tra bài trùng và đếm số yêu cầu còn hiệu lực.

Ba yêu cầu đầu tiên trong cửa sổ thời gian được chấp nhận. Yêu cầu thứ tư hoặc một bài trùng làm tài khoản bị chặn trong ba mươi phút. Kết quả kiểm tra đồng thời trả về toàn bộ activeRequestIds. Backend C sử dụng danh sách này để xóa các bài của tài khoản vi phạm khỏi Max-Heap.

Hash Map giúp truy cập trạng thái tài khoản trong $O(1)$ trung bình. Việc thu hồi k bài khỏi Heap có chi phí $O(k \log n)$, đồng thời duy trì đúng bất biến hàng đợi.

SLIDE 7 - KIẾN TRÚC VÀ LUỒNG TRẠNG THÁI - 60 GIÂY

Giao diện React chỉ nhận sự kiện và hiển thị snapshot. Khi triển khai trên NAS, mọi lệnh đi qua api.php; PHP chỉ là reverse proxy, không lưu queue, revision hay state riêng. Trong môi trường Vite, proxy phát triển chuyển cùng các route đó thẳng tới tiến trình C trên cổng 8081.

Backend C là nguồn sự thật duy nhất. Tại đây, Max-Heap, Map chỉ số, Circular Doubly Linked List và SpamGuard trực tiếp thực hiện request, vote, extractMax, Next, Previous, xóa và trộn hàng đợi. React định kỳ nhận một snapshot kết hợp gồm queue và player; cơ chế sequence loại bỏ phản hồi cũ đến trễ. Nhờ đó giao diện không thể ghi đè trạng thái thuật toán của C.

SLIDE 8 - LUỒNG YÊU CẦU, BÌNH CHỌN VÀ PHÁT BÀI - 90 GIÂY

Sơ đồ này ghép ba cấu trúc dữ liệu thành một luồng xử lý hoàn chỉnh. Khi sinh viên chọn bài, hệ thống kiểm tra điều kiện giấy phép trước. Nếu bài chưa được duyệt, không cho phép phát công cộng hoặc thiếu đường dẫn giấy phép, yêu cầu bị từ chối và sự kiện được ghi vào nhật ký.

Nếu quyền phát hợp lệ, SpamGuard kiểm tra StudentID. Trường hợp hợp lệ, yêu cầu được thêm vào Max-Heap hoặc lượt bình chọn được cập nhật; heapify khôi phục thứ tự ưu tiên. Trường hợp gửi trùng hoặc vượt ba yêu cầu trong mười phút, tài khoản bị chặn ba mươi phút, các requestId đang sở hữu bị thu hồi khỏi Heap và sự kiện được ghi lại.

Khi quản trị viên nhấn Next, extractMax lấy bài ở gốc Heap. Bài vừa phát được nối vào cuối Circular Doubly Linked List bằng addLast, sau đó nhật ký và giao diện được đồng bộ. Điểm quan trọng là mọi kiểm tra đều diễn ra trước khi thay đổi cấu trúc dữ liệu, nên Heap và lịch sử phát luôn giữ đúng bất biến.

SLIDE 9 - KIỂM SOÁT QUYỀN PHÁT - 50 GIÂY

Một bài chỉ được yêu cầu khi trạng thái approved, publicPlaybackAllowed và licenseUrl đều hợp lệ. Giao diện vô hiệu hóa thao tác không đủ điều kiện; backend C kiểm tra lại mã bài và dữ liệu yêu cầu trước khi thay đổi cấu trúc.

Gói phát hành sử dụng mười tám bản piano MP3 do chủ dự án cung cấp. Thông tin nguồn và ghi công được lưu cùng danh mục. HeapBeat không tải, lưu đệm hoặc chuyển đổi nội dung từ Spotify, Apple Music hay YouTube.

SLIDE 10 - GIAO DIỆN QUAN SÁT CẤU TRÚC DỮ LIỆU - 50 GIÂY

Giao diện gồm danh sách phòng, trình phát, hàng đợi ưu tiên và danh mục bài hát. Phần tử đầu Heap được tô nổi bật nên có thể đối chiếu trực tiếp giữa cấu trúc dữ liệu và bài kế tiếp. Điểm số, người yêu cầu, lịch sử phát và trạng thái giấy phép đều được hiển thị tại đúng ngữ cảnh thao tác.

SLIDE 11 - KIỂM THỬ VÀ KẾT QUẢ - 80 GIÂY

Bộ kiểm thử gồm hai mươi hai trường hợp và điều kiện. Circular DLL có bốn trường hợp cho danh sách rỗng, một node và vòng hai chiều. QueueMaxHeap có bảy trường hợp cho thêm, lấy gốc, thay đổi điểm, phân giải bằng điểm và xóa tùy ý.

SpamGuard có năm trường hợp cho hạn mức, bài trùng, thời gian hết chặn và danh sách bài cần thu hồi. Bốn trường hợp tiếp theo kiểm tra trạng thái phát; hai trường hợp cuối xác nhận đủ mười tám tệp MP3 và đường dẫn hợp lệ.

TypeScript được kiểm tra kiểu tĩnh, Vite tạo bản dựng production thành công và backend C vượt qua năm mươi sáu assertion với mọi cảnh báo được xem như lỗi.

SLIDE 12 - LÝ DO LỰA CHỌN MAX-HEAP - 80 GIÂY

Nếu dùng mảng chưa sắp xếp, thêm bài có chi phí $O(1)$ nhưng tìm bài tốt nhất tốn $O(n)$. Nếu giữ mảng luôn sắp xếp, xem đầu là $O(1)$ nhưng mỗi lần thêm hoặc đổi vote có thể dịch $O(n)$ phần tử. Cây cân bằng cũng đạt $O(\log n)$ nhưng phức tạp hơn nhu cầu chỉ lấy phần tử lớn nhất.

Max-Heap kết hợp Map chỉ số cho phép xem gốc $O(1)$, thêm và đổi bình chọn $O(\log n)$, đồng thời tìm vị trí bài trong $O(1)$ trung bình. Đây là lựa chọn cân bằng giữa hiệu năng, khả năng kiểm chứng và độ phức tạp cài đặt.

SLIDE 13 - KẾT LUẬN - 80 GIÂY

HeapBeat hiện thực một Collaborative DJ Queue nhất quán bằng ba cấu trúc dữ liệu. Max-Heap quyết định bài tiếp theo và tự tái cân bằng sau bình chọn. Danh sách liên kết đôi vòng hỗ trợ Next, Previous, Repeat All và thêm cuối. Hash Map theo StudentID phát hiện yêu cầu thứ tư trong mười phút hoặc bài trùng, chặn tài khoản ba mươi phút và thu hồi toàn bộ bài đang chờ.

Mã nguồn backend C, năm mươi sáu assertion C và hai mươi hai kiểm thử giao diện xác nhận các hành vi trên.

Thầy cô có thể quét mã QR trên slide để mở bản demo. Sau đây em xin trình diễn ba luồng chính: bình chọn đổi gốc Heap, chuyển bài bằng danh sách vòng và chặn tài khoản spam.

III. TRÌNH DIỄN HỆ THỐNG - 5 PHÚT

0:00 - 0:35 | MỞ WEBSITE

Mở <https://phamhungtien.synology.me/heapbeat/> và đăng nhập bằng tài khoản demo. Giới thiệu ba vùng chính: trình phát, hàng đợi Max-Heap và danh mục bài hát. Bài được tô nổi bật ở hàng đợi là phần tử gốc Heap.

0:35 - 1:35 | UPVOTE LÀM THAY ĐỔI GỐC MAX-HEAP

Upvote bài ở hạng hai cho tới khi điểm vượt bài hạng nhất. Thứ hạng thay đổi ngay vì Map định vị index, điểm tăng và heapifyUp khôi phục bất biến trong $O(\log n)$.

1:35 - 2:25 | NEXT, PREVIOUS, REPEAT ALL VÀ ADDLAST

Nhấn Next để lấy bài ở gốc Heap và đưa bài vừa phát vào cuối lịch sử. Tiếp tục thực hiện Previous, Next và Repeat All để minh họa liên kết hai chiều cùng liên kết từ đuôi về đầu.

2:25 - 3:50 | CHẶN SPAM VÀ THU HỒI BÀI

Đăng nhập bằng tài khoản sinh viên demo. Gửi trùng một bài hoặc gửi yêu cầu thứ tư trong cửa sổ mười phút. Hệ thống hiển thị thời gian chặn ba mươi phút. Trong giao diện quản trị, toàn bộ bài đang chờ của tài khoản vi phạm đã được xóa khỏi Max-Heap.

3:50 - 4:25 | KIỂM TRA QUYỀN PHÁT

Mở chi tiết một bài đã duyệt và trình bày nguồn, trạng thái phê duyệt, quyền phát công cộng cùng thông tin ghi công.

4:25 - 5:00 | KẾT QUẢ KIỂM THỬ

Trình bày kết quả `npm run test` và `npm run test:backend:c`. Hai mươi hai kiểm thử TypeScript đều đạt; backend C kết thúc với thông báo năm mươi sáu assertion đã thành công.

Em xin kết thúc phần trình bày. Em sẵn sàng trả lời câu hỏi của thầy cô.